部分（主要是第5，6，10章）

第5章 进程API

fork() 以及返回值

wait()、waitpid().

exec() 函数，为什么设计exec()函数？（书P32）

△ prompt > wc p3.c > newfile.txt 具体执行流程？（书P32）

· 如何调用fork(), exec()，中间干了什么.

△ Unix管道的实现，pipe()系统调用？（书P33）

进程间通信kill()函数，ps命令等（书P34）

第6章 受限直接执行 ☆

两个关键点：性能，控制权.

用户模式，内核模式，系统调用过程调用，陷阱（返回）指令

初始化陷阱表.

受限执行操作时 $\xrightarrow{\text{用户 内核}}$ 将寄存器保存到内核栈.

用户 内核 从内核栈恢复到寄存器 $\xleftarrow{}$

进程间切换

协作方式：等待系统调用

非协作方式：操作系统控制. $\boxed{\text{时钟中断}}$

（由OS负责配置时钟中断处理程序并开启时钟）

保存与恢复上下文：（书P43→表6.3受限直接执行协议（时钟中

☆两种类型的reg保存/恢复、

① 时钟中断时，运行进程的 $\boxed{\text{用户reg}}$ 由硬件隐式保存在该进程的内核栈上

② OS决定从A切换到B时，$\boxed{\text{内核reg}}$ 被OS明确保存在该进程的进程结构内存中

保存对象包括通用寄存器、程序计数器，内核栈指针

第10章 多处理器调度 ☆

多处理器架构：缓存一致性问题，缓存亲和度问题，同步
(P75-76)

单队列调度（Single Queue Multiprocessor Scheduling, SQMS）

缺点：扩展性不好（同步开销有限），缓存亲和度差.

多队列调度（Multi-Queue Multiprocessor Scheduling, MQMS）

优点：SQMS的缺点都解决了

缺点：负载不均.（解决方法：迁移，"工作窃取"）

Linux多处理器调度程序.

O(1) 调度　基于优先级(类似MLFQ) ⎫ 多队列
CFS：Completely Fair Schedule　确定的比例调度 ⎭

BF Schedule　比例调度⇐单队列

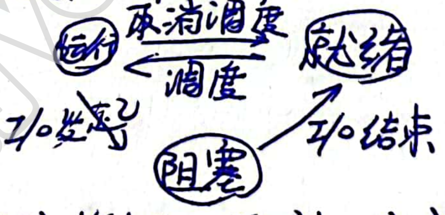
其它的一些点（本次应该不考）

OS设计目标：抽象，高性能，保护，可靠性，安全性（书P11）

OS简单历史：库→库(保护)→多道程序时代→现代　Unix→Lin

进程抽象（进程是什么：书的P20）

进程状态.



进程调度．调度指标、T周转＝完成－到达.
T响应＝首次运行－到达.

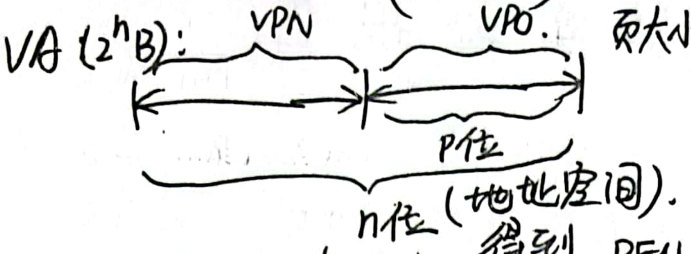
5个基本规则 ↓

FIFO，SJF，STCF(抢占式SJF)，RR，MLFQ.
(multi-level feedback que

比例份额：彩票/步长调度
以及未能广泛使用的原因.

内存管理

▷ 页表是什么作用？(书P133) 页表存在哪里？(书P135) 内存中.



页大小 $P=2^p$.

VPN检索页表 $\xrightarrow{得到}$ PFN(PPN)

页表里有什么？(书P135) 有PFN、有效位、保护位、脏位、参考位……

知VA, PA, P，求每项PTE至少多少位(题目)

▷ TLB (translation-lookaside buffer)

TLB未命中：硬件全权处理.

TLB有效位≠页表有效位 (书P147). TLB有效位只指出该映射是否有效.

TLB交换策略 (LRU要掌握：最近没有用就换出).

▷ 多级页表.

页目录项 PDE (至少拥有有效位与PFN).

页表的每部分都能放入一个页：PTE大小×每级PTE个数 = 2^{VPO}

地址转换过程：优先TLB $\xrightarrow{不命中}$ 多级页表查找并更新TLB (此时再访TLB会命中)

一级页表会因此时间为：2TLB + 2访内 + 页故障处理

正常情况为 TLB+访内 (TLB命中) / TLB+访内(PTE)+TLB+访内 (TLB不命中)

▷ 物理内存机制.

页错误是什么？(书P167). 待访问虚拟地址无物理帧(被交换到磁盘上). 可能杀死进程

⊗存在位，有效位 (书P169)

有效位0 → 非法访问，OS介入

有效位1 存在位0 页错误，换入

存在位1 读取PFN返回.

页交换何时真正发生？(书P169)

▷ 物理内存交换策略.

程序的平均内存访问时间 $AMAT = P_{hit} \cdot T_{memory} + P_{miss} \cdot T_{Disk}$.

最优替换策略：替换内存中在最远将来才会被访问到的页. (只作比较).

FIFO / 随机Random.

LRU (Least Recently Used) / LFU (Least Frequently Used) / MFU / MRU

时钟算法. (书P181)

脏页与脏位 (倾向于踢出干净页)

抖动. (书P183)

并发

核心问题：不可控的调度，不合时宜的中断.

竞态条件，临界区，原子性，互斥原语.硬件同步原语（书P206）

线程API（要会写一些代码）.

①创建线程与线程完成.

```
#include<pthread.h>
void* mythread(void* arg){…} 这是线程执行的函数.
int main(){ int m; //希望的返回值
    pthread-t p;
    (int rc =)pthread_create(&P,NULL,mythread,&args);
    pthread-join(p,(Void**)&m);
    return 0;}.
```

注意.mythread不要返回栈上指针.

②锁.

```
pthread_mutex-t lock;
(int rc=)pthread_mutex-init(&lock,NULL);
~~mute~~ pthread-mutex-lock(&lock);
      ……
pthread- mutex_unlock(&lock);
```

③条件变量（使用条件变量时必须持有锁）

```
pthread_mutex_lock(&lock);                     } 等待者.
while(ready==0)
    pthread-mutex_wait(&cond,&lock);
pthread_mutex-unlock(&lock);
pthread-mutex-lock(&lock);                     } 唤醒者.
ready=1;
pthread-mutex-signa((&cond);
pthread-mutex-unlock(&lock);
```

锁(28章)

自旋锁 void init(lock_t *lock){ lock→flag = 0; //可用了;

→void lock(lock_t *lock){ while(TestAndSet(&lock→flag, 1)==1); }

void unlock(lock_t *lock){ lock→flag=0; //可用了;

①TestAndSet：设置new并返回old.(原子性)

②CompareAndSwap：若与预期相符，修改为new.

→void lock(lock_t *lock){while(CompareAndSwap(&lock→flag, 0, 1)==1);};

③LL/SC (书P227)

④FetchAndAdd：返回旧值并自增. (书P228)

休眠队列. guard, flag, queue_t *q;

持有guard才可操作后两个,有flag才进临界区,否则进队列.

释放时队列空就释放flag,否则unpark()。

(书P231).

两阶段锁 (书P233).

条件变量(30章)

父线程等待子线程问题. (书P251)

1. 调度2种情况，父先子后，子先父后，分别怎么样？大体描述
2. 如果没有done会怎么样？(父运行时,不知子状态)
3. 发信号与等待时都不加锁会怎么样？(长眠不醒)

※发信号时总持有锁!!! pthread_mutex_(un)lock(&m).

△声明条件变量 pthread_cond_t c;

pthread_cond_wait(&c, &m)

pthread_cond_signal(&c)

※生产者/消费者(有界缓冲区)问题. →Mesa语义.

1. 使用if(count=0或1)作为wait语句的判断有什么问题？怎么解

书P255 (消费者1在wait上被唤醒，但消费者2从不在wait上并抢先)

2. 使用同一个信号量&cond有什么问题？怎么解？

书P257 (消费者1消费完后唤醒消费者2，最终都沉睡) →信号

只有

3. 若为2个生产者，1个消费者，上述问题如何构造？
4. 若为1个生产者，1个消费者，是否存在上述问题？
5. 最终的单值缓冲区及多值缓冲区方案？

覆盖条件

pthread_cond_broadcast() 唤醒所有线程.

(可解决上面的哪个问题？(2))

信号量（31章）

代码实现：

```
#include <semaphore.h>
sem-t  s;
sem-init(&s,0,1); //初始化值为1

sem_wait(&m);     // P( ), down  #调用时≥1,直接返回;否则挂起
sem-post(&m);     // V( ), up.   #信号量+1;若有等待的就唤醒一个.
```

信号量值 <0 时，abs( )为等待线程个数.

信号量实现锁（二值信号量）.

```
sem-t m;
sem-init(&m,0,1);
 ......
sem_wait(&m);
 临界区.
sem_post(&m);
```

信号量实现条件变量

父线程等待子线程问题.

```
child: sem-post(&S)
parent: pthread-t c;
        int rc=pthread-create(c,NULL,child,NULL);
        sem_wait(&s);
```

在最开始 sem-init(&s,0,?) ?=0 为什么？

考虑不同调度，父先子后怎么样，子先父后怎么样？

临界缓冲区问题

1. 直接用~~条件变量~~信号量将临界区环绕有什么问题？怎么解？（书P269）

（临界区>1时，两个~~消费~~者依次访问~~将导致~~若调度不巧将导致~~覆~~盖 [illegible] 先生产者的数据）

（同理，两个消费者也会~~读~~可能读取同一片区域）

2. 使用锁将条件变量环绕有什么问题？怎么解？（书P271）

（死锁） 缩小mutex锁的作用域.

3. sem-init 怎么写.

4. 正确版本怎么写.

读者、写者锁. （书P272）

哲学家就餐问题.

HNU计算机学习使用

并发部分可能的简答题（小知识点）

1. [锁] 如何评价锁？ P220
2. [锁] 为什么不采用临界区关闭中断来实现锁（4点）？ P221 -
3. [锁] 请按照评价锁的方法评价自旋锁。 P225
4. [锁] 实现自旋锁的硬件原语有哪些，并简要分析。（4点） 第28章

常见并发问题

分为 非死锁缺陷 和 死锁缺陷

非死锁缺陷

违反原子性缺陷 （解决方法：加锁）

违反顺序缺陷. （解决方法：加锁/条件变量）

指违反了预期的访问顺序.

死锁缺陷

4个必要条件：（书P284）互斥，持有并等待，非抢占，循环等待.

死锁的预防，避免（通过调度），检查和恢复.

HNU甘晴void学习使用

管理与文件系统

磁盘驱动器

旋转延迟，寻道时间，传输时间.

磁道偏斜，磁道缓冲区，后写，直写.

RPM，最大传输速度，平均寻道时间，相关计算

磁盘调度算法

SSTF (Shortest-Seek-Time-First) 饥饿问题

电梯 (SCAN/C-SCAN) 未遵循SJF，忽视旋转.

SPTF (Shortest-positioning TimeFirst) 最短接入时间优先.

RAID ☆

做计算题，RAID，disk，offset，编号都是从0开始的.

如何评估RAID？容量，可靠性，性能.

RAID 0：条带化：在对数组连续块请求时可获最大并行性.

△ 大块大小与思考：大块↓ → 文件跨多个磁盘 → 对单个文件读写并行性↑(优)
定位时间↑(劣)

RAID 1：镜像：顺序读时，跳过的不会提供有效带宽.

RAID 4：奇偶校验：每行1的个数为偶.

顺序读写：全条带写入 → 顺序写，是对RAID 4最有效的方式

随机读写：

加法奇偶校验.

减法奇偶校验：$P_{new} = (C_{old} \oplus C_{new}) \oplus P_{old}$.

基于奇偶校验的RAID的小写入问题
(奇偶校验盘不会实现并行)
瓶颈在于对于每逻辑I/O产生2次I/O.

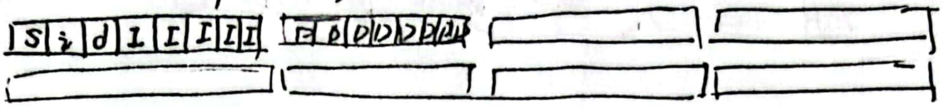
RAID 5：旋转奇偶校验.

总结(书P337)		RAID-0	RAID-1	RAID-4	RAID-5
容量					
可靠性					
吞吐量					
	顺序读				
	顺序写				
	随机读				
	随机写				
延迟	读				
	写				

文件系统

数据结构与访问方法　心智模型.

整体组织：Super块，inode/data位图，inode块，Data块.



inode（保存关于文件的信息）

直接指针，间接指针（指向全是直接指针/间间接指针的块）

可构建多级索引.（多重间接指针）.

关于文件系统的一些特点.

大多数文件很小，少部分大文件用了大部分空间.

详见例题与考题再总结.

目录组织.

.（当前目录）　..（父目录）

目录被视为特殊的文件.

空闲空间管理.

HNU甘晴void-仅供学习使用

RAID 比较：总结

现在简单总结一下表 38.10 中各级 RAID 的比较。请注意，我们省略了一些细节来简化分析。例如，在镜像系统中写入时，平均查找时间比写入单个磁盘时稍高，因为寻道时间是两个寻道时间（每个磁盘上一个）的最大值。因此，对两个磁盘的随机写入性能通常会比单个磁盘的随机写入性能稍差。此外，在 RAID-4/5 中更新奇偶校验磁盘时，旧奇偶校验的第一次读取可能会导致完全寻道和旋转，但第二次写入奇偶校验只会导致旋转。

表 38.10　RAID 容量、可靠性和性能

	RAID-0	RAID-1	RAID-4	RAID-5
容量	N	$N/2$	$N-1$	$N-1$
可靠性	0	1（肯定）		
		$N/2$（如果走运）		
吞吐量				
顺序读	$N \cdot S$	☆ $(N/2) \cdot S$	$(N-1) \cdot S$	$(N-1) \cdot S$
顺序写	$N \cdot S$	$(N/2) \cdot S$	$(N-1) \cdot S$	$(N-1) \cdot S$
随机读	$N \cdot R$	$N \cdot R$	$(N-1) \cdot R$	$N \cdot R$
随机写	$N \cdot R$	$(N/2) \cdot R$	☆ $1/2 \cdot R$	☆ $N/4 \cdot R$

都不并行，
修改位/校验位

可并行
但四倍操作.